Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 3b - Decision Making as Supervised Learning (part 2)

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Recap: Supervised Learning Setup

Behavior Cloning
Active Learning
Conclusion

Input: A finite data set generated from an unknown distribution $(x, y) \sim \mathcal{D}$ over inputs and labels, and a hypothesis class H

Objective : finding a hypothesis that minimizes the expected loss $h^\star = \arg\min_{h \in H} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\ell(h(x), y)\right]$

Empirical Objective: We approximate this objective using the empirical loss:

$$\hat{h} = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

Behavior Cloning

Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion

Imitation learning is a family of methods where an agent learns to act by observing an expert.

Behavior cloning is the simplest form of imitation learning, where the objective is to learn a policy by mimicking the behavior demonstrated by an expert.

A policy is trained to minimize the discrepancy between its predicted actions and the expert's actions over the collected dataset.

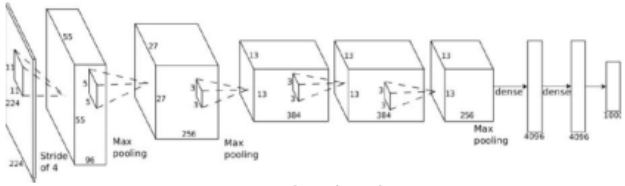


Behavior Cloning: Example

Behavior Cloning
Active Learning
Conclusion



$\mathbf{o}_t$



$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$



$\mathbf{a}_t$



$\mathbf{o}_t$
$\mathbf{a}_t$



$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

Behavioral cloning

Can be broken down into the following steps:

- **Data Collection:** Gather a dataset of state-action pairs (s_i, a_i), where s_i represents the state and a_i represents the action taken by the expert at that state.
- **Model Selection:** Choose a model to approximate the expert's policy. Typically, a neural network $\pi_\theta(s)$ parameterized by θ.

Ideas for managing the learning process?

Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion

Given expert demonstrations $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{m}$, we train a policy to predict the expert's action.

Loss measures the *discrepancy* between the agent's predicted actions and the expert's actions over the collected dataset.

How do we define loss?

Behavioral Cloning

Continuous actions: Squared-error loss

For actions such as steering angles or joint velocities:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \underbrace{\|a_i - \pi_\theta(s_i)\|^2}_{\text{squared action prediction error}} .$$

Here, $\pi_\theta(s_i)$ outputs an action vector.

Meaning?

Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion

Discrete actions: Cross-entropy loss

For actions such as *left*, *right*, or *stop*:

$$\hat{\mathcal{R}}(\theta) = -\frac{1}{m}\sum_{i=1}^{m} \log \pi_\theta(a_i \mid s_i).$$

Here, $\pi_\theta(a_i \mid s_i)$ is the probability assigned to the expert's action.

Meaning?

Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion

Training: Optimize the model parameters θ by minimizing the empirical risk (e.g., using gradient descent)

$$\theta^* = \arg\min_{\theta} \hat{\mathcal{R}}(\theta)$$

Policy Execution: Use the trained model $\pi_{\theta^*}(s)$ as the policy to predict actions for new states during execution.

Strengths? Limitations?

Strengths of Behavioral Cloning

- **Simple and intuitive:** reduces policy learning to standard supervised learning
- **Data-driven:** can learn directly from expert demonstrations without specifying a reward function
- **Efficient training:** can leverage existing optimization methods and large supervised datasets
- **Useful when expert behavior is available:** especially effective when demonstrations are abundant and high quality
- **Good initialization:** often provides a strong starting point for further policy improvement

Behavior cloning makes sequential decision-making accessible through standard supervised learning tools.

Supervised Learning Assumption

We learn a hypothesis by minimizing empirical risk:

$$\hat{h} = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

Key assumption: data is sampled i.i.d. from a fixed distribution

$$(x, y) \sim \mathcal{D}_{\text{train}} = \mathcal{D}_{\text{test}}$$

Implication

Generalization guarantees hold only under this assumption.

Limitations of Behavioral Cloning

In many settings, the IID assumption breaks because:

- Decisions are sequential: predictions are used to make decisions, which affect future inputs.
- The data distribution changes, i.e.,

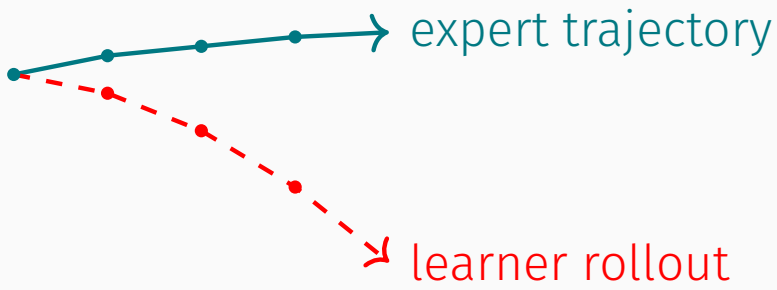
$$\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{test}}$$

The difference between the training and test distributions is called: **distribution shift**

- Train on one distribution, deploy on another
- Model encounters inputs it has never seen before
- Generalization guarantees no longer apply

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion



Distribution Shift: The agent follows π_θ trained on (s, a) from expert demonstrations, but encounters states that were never demonstrated.

Key challenge: Errors compound over time.

- Small prediction error leads to a slightly wrong action
- The system transitions to a new, unseen state
- Errors increase as the agent moves further from expert behavior

Effect: performance degrades rapidly.

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion



In autonomous driving, a small steering mistake moves the car off the lane, a situation rarely observed in expert data.

DAgger: Addressing the Distribution Shift

Behavior Cloning
Active Learning
Conclusion

Key idea: train on the states the learner actually visits

- Run current policy π_i to collect states $s \sim \mathcal{D}^{\pi_i}$
- Query expert for correct actions a^*
- Aggregate dataset:

$$D \leftarrow D \cup \{(s, a^*)\}$$

- Retrain policy on updated dataset

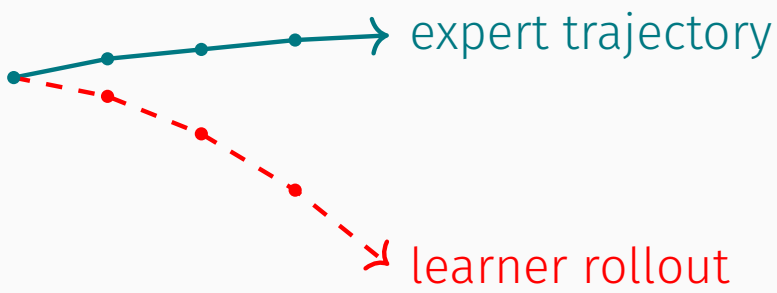
DAgger (Dataset Aggregation)

Behavior Cloning
Active Learning
Conclusion

Dataset Aggregation (DAgger)

1: Initialize dataset $\mathcal{D} = \{(o_1, a_1), \ldots, (o_N, a_N)\}$
2: **for** iterations **do**
3: Train policy $\pi_\theta(a_t \mid o_t)$ on $\mathcal{D}$
4: Collect observations $\{o_1, \ldots, o_M\}$ using π_θ
5: Query expert for labels $\{a_1^*, \ldots, a_M^*\}$
6: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_1, a_1^*), \ldots, (o_M, a_M^*)\}$
7: **end for**

DAgger



DAgger queries the expert on states visited by the learner, and adds these corrections to the training set.

What assumptions are made here about access to the expert during training?

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion





Is this the same observation?

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion





Is this the same observation?

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion





Is this the same observation?

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion





Is this the same observation?

Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion

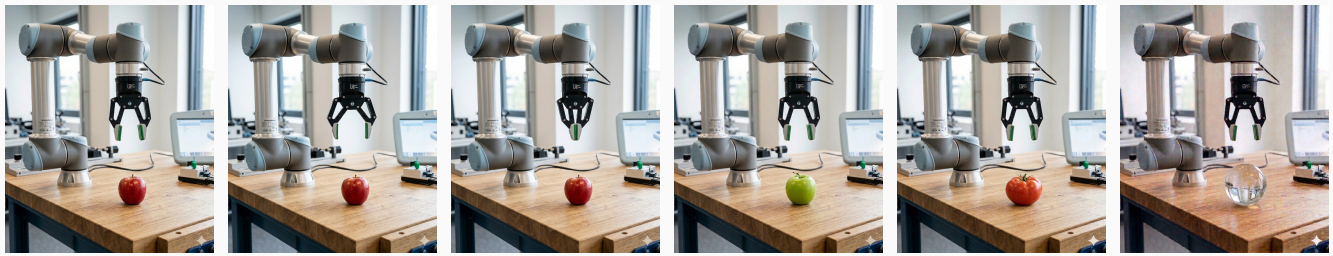




Is this the same observation?

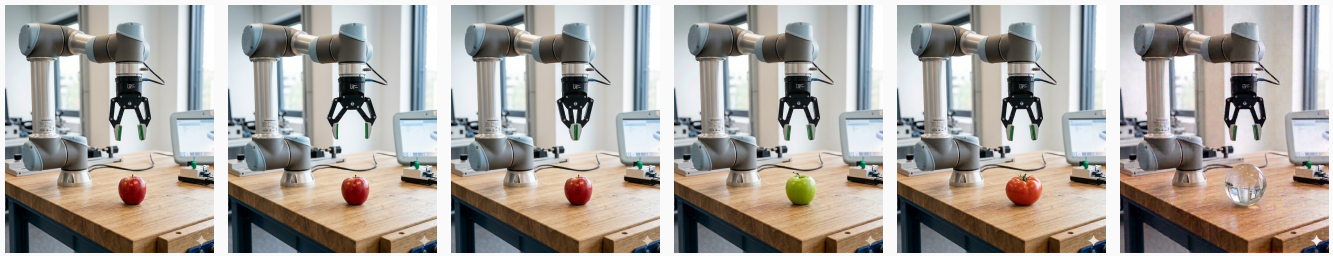
Limitations of Behavioral Cloning

Behavior Cloning
Active Learning
Conclusion



BC assumes a consistent mapping from observations to actions - this breaks under ambiguity and distribution shift.

Limitations of Behavioral Cloning



State aliasing \Perceptual aliasing (POMDP view): different underlying states produce visually similar observations, but require different actions:

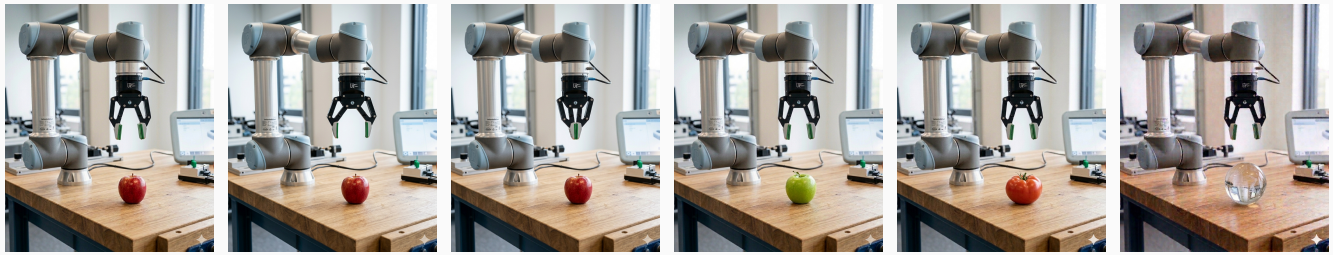
$$o \approx o' \quad \text{but} \quad a^\star \neq a^{\star\prime}$$

In behavior cloning, the policy is learned as a direct mapping from observations to actions, i.e., $\pi(a \mid o)$.

In this case, conflicting supervision is recevied and classical methods tend to learn the **average action**: $\mathbb{E}[a \mid o]$ which can be suboptimal or even invalid.

Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning



The opposite can also occur: The same underlying state may appear visually different, and therefore be treated as out-of-distribution.

Covariate shift \Observation variability: The same state appears differently across contexts:

$$s = s' \quad \text{but} \quad o \neq o'$$

→ *observations fall outside training distribution*

Behavior Cloning: Key Takeaways

Behavior Cloning works well when:

- Observations are **informative** (no state aliasing)
- The action distribution is **unimodal** (a single action per state)
- Train and test distributions are **aligned**

Behavior Cloning often fails with:

- **Distribution shift:** $\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{test}}$
- **State aliasing:** $o \approx o'$ but $a^\star \neq a^{\star\prime}$
- **Multimodality:** $\pi(a \mid o)$ has multiple valid modes

Key insight: Behavior cloning reduces decision-making to supervised learning — but **sequential structure and partial observability break its assumptions.**

Motivates interactive and sequential methods (DAgger, RL,...)

Active Learning

From Passive to Active Learning

Standard supervised learning is passive.

- The dataset is given in advance
- The learner does not choose which examples appear
- The learner does not choose which labels are acquired

In many domains, labeled samples are expensive or rare.

- human annotation requires time and expertise
- experiments may be costly
- inspections may require physical access
- high-quality measurements may consume resources

From Passive to Active Learning

Examples:

- annotating medical images by doctors
- labeling road scenes for autonomous driving
- performing laboratory experiments
- collecting high-quality sensor measurements






Which example should we label next?

This question leads from **supervised learning** to **active learning**.

Active Learning

Behavior Cloning
Active Learning
Conclusion

Two primary setups:

- **Pool-based:** The model selects samples from a large unlabeled pool of data.
  For example, the most uncertain texts from a large pool to ask a human annotator to label.
- **Stream-based:** The model receives samples sequentially (one sample at a time) and decides whether to label them. The data is gone if the decision maker decides not to label it.

    - For example, a system monitoring sensor data decides on-the-fly whether new sensor readings are valuable enough to label.

We will focus on the former, but many of the insights and decision-making principles apply to both settings.

Active Learning

Behavior Cloning
Active Learning
Conclusion

Active learning allows the learner to **actively select** the most informative examples to label, based on information it can acquire using a **limited budget**.

Given an unlabeled pool of samples $S = \{x_1, \ldots, x_n\}$ the learner repeatedly:

1. selects a query $x^\star$
2. obtains its label $y^\star$
3. updates the model

Goal: maximize model accuracy given a limited labeling budget.

Active Learning is Not Control nor Meta-Learning

In control and reinforcement learning, actions are chosen to **affect the environment** and maximize cumulative reward.

In active learning, the learner chooses **which information to acquire**, rather than **which control action to execute**. Under the standard formulation, queries do not alter the environment state or data distribution.

In meta-learning, the goal is to **learn how to learn across tasks**, i.e., to acquire an **inductive bias** that enables rapid adaptation to new tasks.

In contrast, active learning focuses on a **single task**, aiming to **improve performance** by selecting informative data, rather than improving the learning procedure itself.

Active Learning is Sequential

Active learning unfolds over time.

At round t:

- labeled dataset: S_t
- unlabeled pool: U_t
- current model: $\Pr_{\theta_t}(y \mid x)$, trained on S_t, parametrized by θ_t

The learner selects:

$$x_t^\star = \arg\max_{x \in U_t} \mathrm{QueryScore}(x; \theta_t)$$

Receives label $y_t^\star$, and updates:

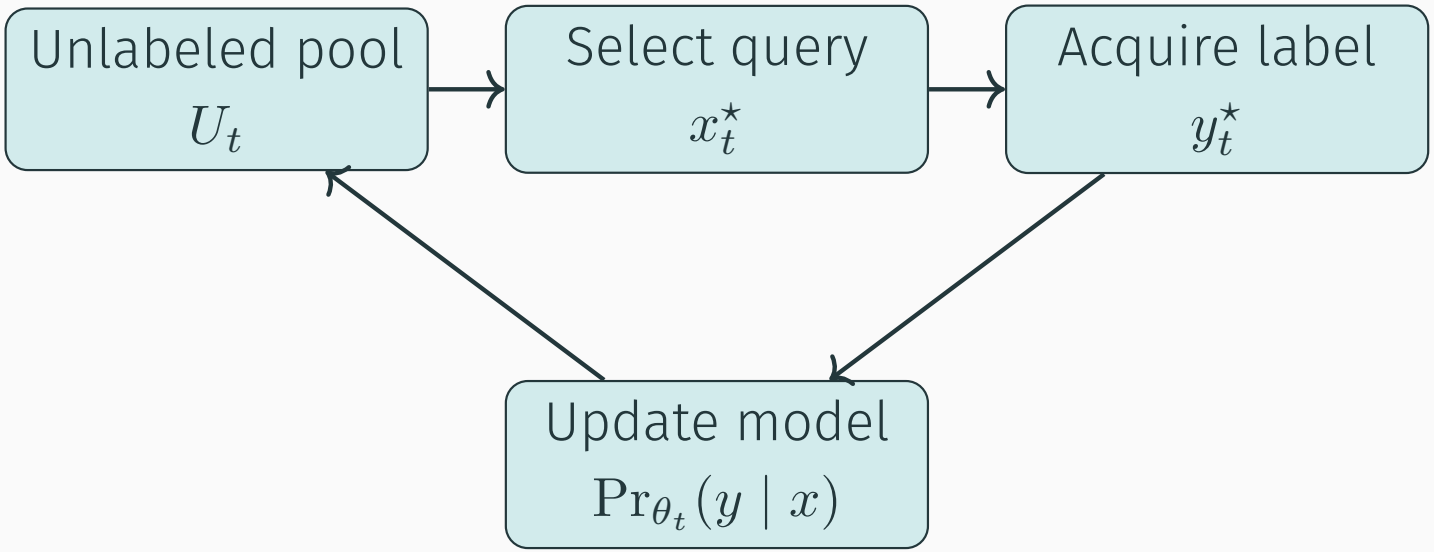
$$S_{t+1} = S_t \cup \{(x_t^\star, y_t^\star)\}$$

$$U_{t+1} = U_t \setminus \{x_t^\star\}$$

$$\theta_{t+1} \leftarrow \mathrm{Train}(S_{t+1})$$

Active Learning Loop

Behavior Cloning
Active Learning
Conclusion



The hypothesis space over the underlying model is represented by θ.

Active Learning

Behavior Cloning
Active Learning
Conclusion

Does active learning aim to address epistemic or aleatoric uncertainty?

Active Learning

Does active learning aim to address epistemic or aleatoric uncertainty?

Epistemic uncertainty (model uncertainty) - can be reduced with more observations.

Aleatoric uncertainty (inherent noise) - cannot be reduced by querying more data.

Active learning is designed to prefer queries that reduce **epistemic uncertainty** and avoid queries dominated by **aleatoric noise**

Choosing the Next Query: value of information

Let $\mathcal{U} \subseteq \mathcal{X}$ is the pool of unlabeled examples.

$$x_{\text{next}} = \arg\max_{x \in \mathcal{U}} \text{QueryScore}(x; \theta)$$

Different strategies define $\text{QueryScore}(.)$, including:

- **Uncertainty sampling:** query the point for which the current model is least certain
- **Query-by-committee:** query points on which several candidate models disagree
- **Expected error reduction:** query the point expected to most reduce future prediction error
- **Expected information gain:** query the point expected to most reduce uncertainty about the model

Reminder: Entropy

Measures the **uncertainty** or **information content** of a random variable. Let X be a discrete random variable taking values in $\mathcal{X}$, distributed according to $p(x) := \Pr(X = x)$

$$\mathcal{H}(X) = \mathbb{E}\big[-\log p(X)\big]$$

Explicitly,

$$\mathcal{H}(X) = -\sum_{x} p(x) \log p(x) \qquad \text{(discrete case)}$$

$$\mathcal{H}(X) = -\int p(x) \log p(x)\, dx \qquad \text{(continuous / differential entropy)}$$

Interpretation for the discrete case: Expected number of bits required to identify the outcome of X.

Conditional entropy: $\mathcal{H}(Y \mid X) = \mathbb{E}\big[\mathcal{H}(Y \mid X = x)\big]$. Measures the remaining uncertainty in Y after observing X.

Reminder: Information Gain

Information gain measures the reduction in uncertainty about one variable after observing another, i.e., how much knowing X reduces uncertainty about Y.

$$I(Y; X) = \mathcal{H}(Y) - \mathcal{H}(Y \mid X)$$

- $\mathcal{H}(Y)$: uncertainty about Y before observing X
- $\mathcal{H}(Y \mid X)$: remaining uncertainty after observing X

Equivalent symmetric form: $I(Y; X) = \mathcal{H}(X) - \mathcal{H}(X \mid Y)$

Alternative expression: $I(Y; X) = \mathcal{H}(Y) + \mathcal{H}(X) - \mathcal{H}(X, Y)$

Choosing the Next Query

Uncertainty sampling: Query the point for which the current model, defined by θ, is least certain,

$$\mathrm{QueryScore}(x; \theta) = \mathcal{H}\big(p_\theta(y \mid x)\big)$$

Here, uncertainty is measured by the entropy of the model's predictive distribution.

Choosing the Next Query

Example: Consider a binary classification problem with two classes y_1 and y_2. We have three unlabeled instances x_1, x_2, x_3 and the corresponding predictive distributions are as follows:

$$p(y_1|x_1) = 0.6, \quad p(y_2|x_1) = 0.4$$
$$p(y_1|x_2) = 0.3, \quad p(y_2|x_2) = 0.7$$
$$p(y_1|x_3) = 0.8, \quad p(y_2|x_3) = 0.2$$

Based on Entropy-based uncertainty sampling - which will be chosen next?

Choosing the Next Query

Example: Consider a binary classification problem with two classes y_1 and y_2. We have three unlabeled instances x_1, x_2, x_3 and the corresponding predictive distributions are as follows:

$$p(y_1|x_1) = 0.6, \quad p(y_2|x_1) = 0.4$$
$$p(y_1|x_2) = 0.3, \quad p(y_2|x_2) = 0.7$$
$$p(y_1|x_3) = 0.8, \quad p(y_2|x_3) = 0.2$$

Based on Entropy-based uncertainty sampling - which will be chosen next?

- $\text{QueryScore}(x_1) = -0.6\log(0.6) - 0.4\log(0.4) = 0.29$
- $\text{QueryScore}(x_2) = -0.3\log(0.3) - 0.7\log(0.7) = 0.27$
- $\text{QueryScore}(x_3) = -0.8\log(0.8) - 0.2\log(0.2) = 0.22$

We would select x_1 for labeling as it has the highest entropy, indicating the model is most uncertain about its prediction at x_1.

Choosing the Next Query

Query-by-committee: Query points on which several candidate hypotheses disagree.

$$\text{QueryScore}(x) = \text{Var}_{h \in H}\left[h(x)\right] \quad \text{or} \quad \text{Disagreement}(x)$$

Maintain a **distribution or set of hypotheses** consistent with current data. Prefer queries where predictions vary the most across models

Approximating **epistemic uncertainty**
via disagreement between plausible models.

Choosing the Next Query

Where do different models come from?

Choosing the Next Query

Where do different models come from?

- **Ensembles:** different random initializations of optimization trajectories $\Rightarrow$ different local minima.
- **Bootstrap resampling:** each model sees a slightly different dataset.
- **Model resampling:** draw different model parameters.

Choosing the Next Query

Expected error reduction: Query the point expected to most reduce future prediction error

$$\mathcal{R}(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\ell(h_\theta(x), y)\big]$$

$$\mathrm{QueryScore}(x) = \mathbb{E}_{y\sim\mathrm{Pr}_\theta(\cdot|x)}\left[\mathcal{R}(\theta) - \mathcal{R}\big(\theta^{+(x,y)}\big)\right]$$

- Measures expected improvement in generalization error
- This requires retraining for each candidate (x, y)
- In practice, the expected change is approximated, e.g., via local updates instead of full retraining.

Choosing the Next Query

Expected information gain: query the point expected to most reduce uncertainty about the model

$$\mathrm{QueryScore}(x) = I\big(y; \theta \mid x\big) = \mathcal{H}(\theta) - \mathcal{H}(\theta \mid y, x)$$

Where $\mathcal{H}(\theta) = -\int p(\theta) \log p(\theta)\, d\theta$

and $\mathcal{H}(\theta \mid y, x) = \mathbb{E}_{y \sim p(y|x)} \left[-\int p(\theta \mid y, x) \log p(\theta \mid y, x)\, d\theta \right]$

What is the connection to aleatoric and epistemic uncertainty?

Choosing the Next Query

Expected information gain:

$$I(y; \theta \mid x) = \underbrace{\mathcal{H}(p(y \mid x))}_{\text{total uncertainty}} - \underbrace{\mathbb{E}_\theta[\mathcal{H}(p(y \mid x, \theta))]}_{\text{aleatoric uncertainty}}$$

- Subtracts irreducible noise
- Focuses on uncertainty about the model

Good queries are those that are uncertain because we don't know the model (epistemic uncertainty)
not because the world is noisy (aleatoric uncertainty)

Choosing the Next Query in Continuous Domains

In continuous input spaces, we cannot evaluate $\arg\max_{x \in \mathcal{X}} \mathrm{QueryScore}(x)$ directly, since $\mathcal{X}$ is infinite.

Key idea: restrict the search to a finite candidate set $\mathcal{C} \subset \mathcal{X}$.

How do we construct $\mathcal{C}$?

- **Random sampling:** sample $x \sim p(x)$ from the input distribution
- **Pool-based setting:** select from a large unlabeled dataset $\mathcal{U}$
- **Grid / discretization:** uniform or adaptive discretization of the space
- **Local optimization:** directly optimize $\mathrm{QueryScore}(x)$ using gradient-based methods
- **Model-driven sampling:** focus on regions of high uncertainty (e.g., near decision boundary)

Active learning as decision-making under uncertainty

Selecting a query x is analogous to taking an information-gathering action.

Connection to Planning and Reinforcement Learning

- Query selection $\leftrightarrow$ action selection
- Labels/observations $\leftrightarrow$ stochastic outcomes
- Model update $\leftrightarrow$ belief update
- Objective $\leftrightarrow$ maximize long-term utility

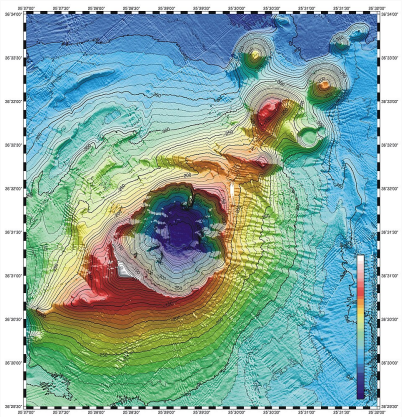
Active learning can be viewed as a myopic POMDP, where queries are actions and information has utility.

$$\mathrm{VOI}(x) = \mathbb{E}_{y \sim p(y|x)}\left[U(\theta \mid \mathcal{D} \cup \{(x, y)\})\right] - U(\theta \mid \mathcal{D})$$

Example: Active Environmental Sensing

A robot wants to learn a spatial field $f(x)$ (e.g., temperature, radiation, terrain cost), but measurements are expensive.

Active learning decides where to measure next.



The robot learns the map efficiently with fewer measurements.

Conclusion

We considered two approach to using supervised learning within and decision making setting: Behavior cloning and active learning.

What is the core difference between the approaches in terms of the use of supervised learning?

Considering each approach to behavior cloning and active learning, we have seen:

- Is it **complete**? Is it **optimal**?
- What are its **time and space requirements**?
- Does it exhibit **anytime behavior**?
- How does it use **resources** (computation, memory, data)?
- Can it handle **uncertainty** or only deterministic settings?